

PLPP Assignment 4

Matvii Kovalyshyn

Git repo link: <https://github.com/VY-Assignments/text-editor-matvii-kovalyshyn>

1. Program Explanation

- **OOP design:** Describe your class hierarchy for Line types (TextLine, ContactLine, ChecklistLine) and how responsibilities are divided between the TextEditor and Cipher classes.

We have a parent class Line. TextLine, ContactLine and CheckListLine inherits it. A list of Lines is stored in TextEditor class. Each class has specific fields and methods for it, and the base class has some virtual methods. TextEditor stores most of the functionality. Caesar and Vigenere classes are children of Cipher class.

- **DLL usage:** Explain how the external cipher library is loaded at runtime (e.g., using LoadLibrary or dlopen) and why it's not stored in RAM continuously.

Cipher library is loaded at runtime using dlopen. It is stored in disk as .so file and loaded only when it is needed in runtime, after that it unloads.

- **Virtual dispatch:** Explain how the print() and serialize() methods resolve to the correct derived class implementation at runtime via the vtable.

In base class Line we have virtual methods Print() and Serialize(). Its override is provided in each subclass.

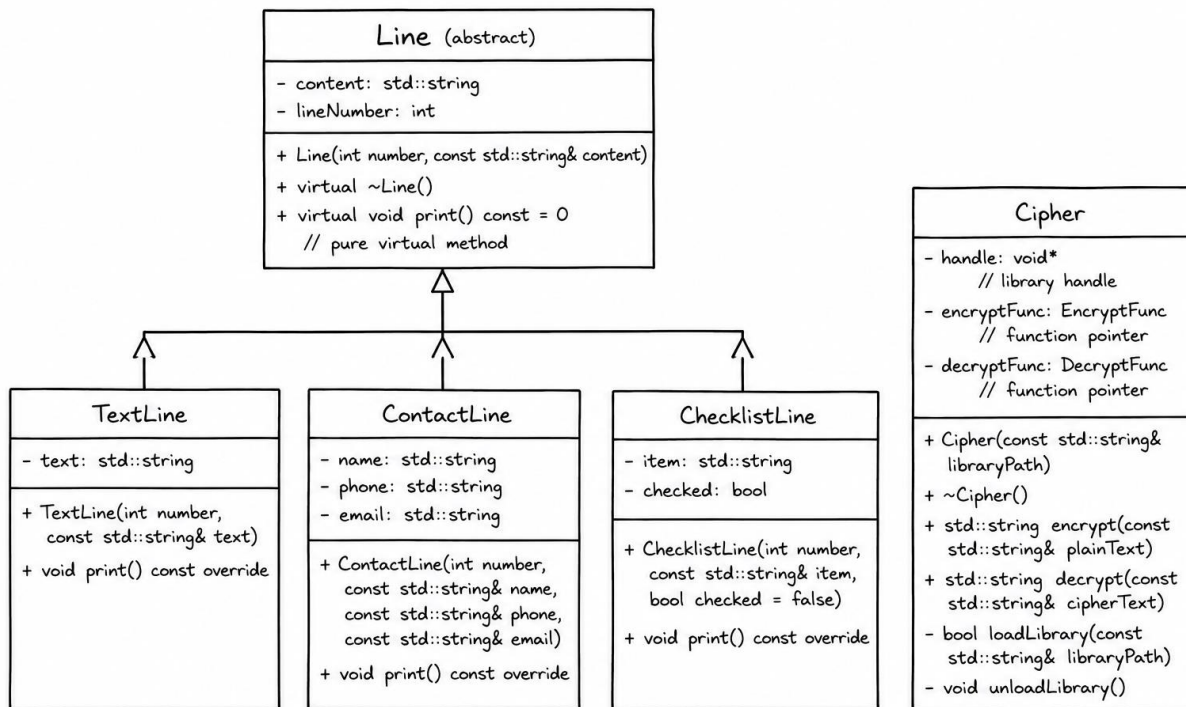
- **Memory management:** Describe which component is responsible for allocating and freeing the dynamic Line objects.

Memory is allocated using new and constructor of a class. Free is using delete and destructor, where lines are deleted first and then the whole class.

2. Class Diagram

Sketch your class hierarchy (hand-drawn or tool-generated). Must show:

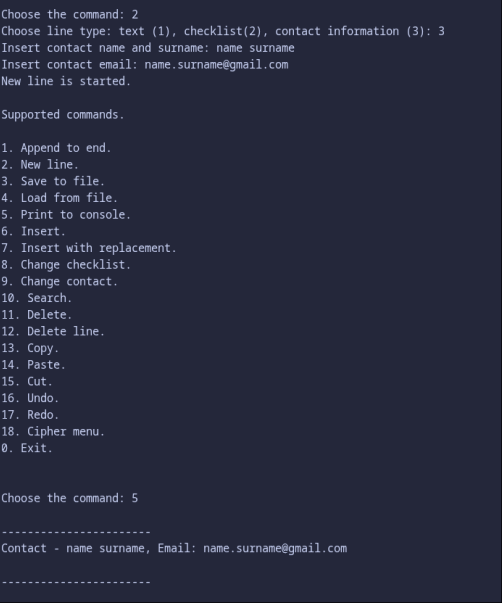
- Line (abstract base class) with print() as a pure virtual method.
- TextLine, ContactLine, and ChecklistLine as derived classes.
- Cipher class encapsulating the dynamic library operations.
- Arrows indicating inheritance relationships.

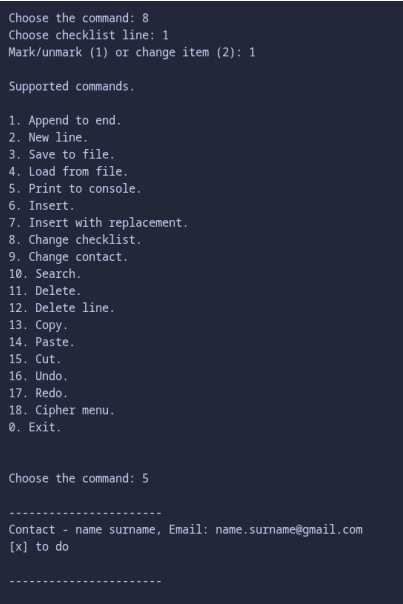


3. Test Plan — One screenshot per scenario

Base Flows

Scenario / Test Case	Expected Result
Add & Print Contact Row	Contact row successfully added and displayed with correct formatting (Name, Surname, E-mail).
Screenshot: <PUT HERE>	

Scenario / Test Case	Expected Result
	
Add & Print Checklist Row	Checklist row successfully added and displayed with an empty checkbox [].
Screenshot: <PUT HERE> 	
Toggle Checklist Status	Changing a checklist item status updates the display to [x] correctly.
Screenshot: <PUT HERE>	

Scenario / Test Case	Expected Result
 <pre> Choose the command: 8 Choose checklist line: 1 Mark/unmark (1) or change item (2): 1 Supported commands. 1. Append to end. 2. New line. 3. Save to file. 4. Load from file. 5. Print to console. 6. Insert. 7. Insert with replacement. 8. Change checklist. 9. Change contact. 10. Search. 11. Delete. 12. Delete line. 13. Copy. 14. Paste. 15. Cut. 16. Undo. 17. Redo. 18. Cipher menu. 0. Exit. Choose the command: 5 ----- Contact - name surname, Email: name.surname@gmail.com [x] to do ----- </pre>	
Polymorphic Save & Load (Serialization)	Saving a file with mixed line types (text, contact, checklist) and reloading it preserves all types and contents perfectly.
Screenshot: <PUT HERE>	



```

Choose the command: 5

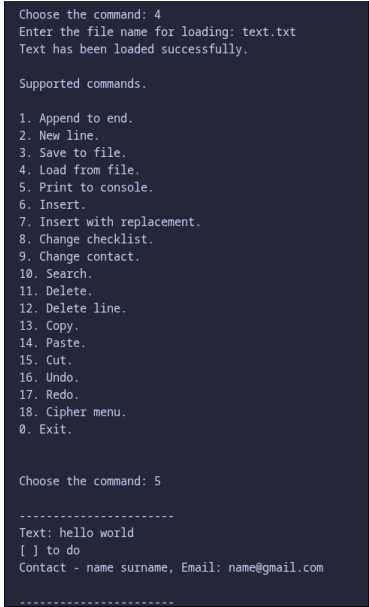
-----
Text: hello world
[ ] to do
Contact - name surname, Email: name@gmail.com
-----

Supported commands.

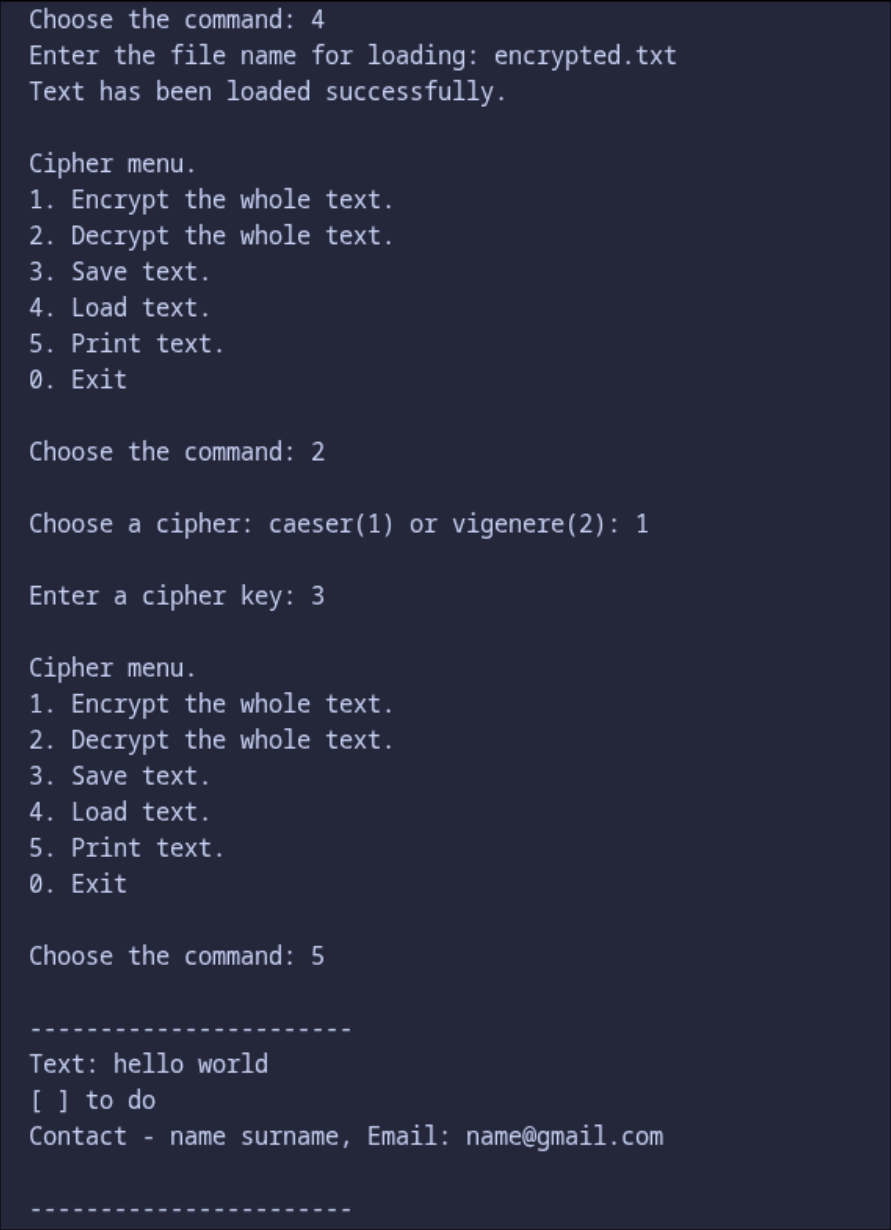
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Insert with replacement.
8. Change checklist.
9. Change contact.
10. Search.
11. Delete.
12. Delete line.
13. Copy.
14. Paste.
15. Cut.
16. Undo.
17. Redo.
18. Cipher menu.
0. Exit.

Choose the command: 3
Enter the file name for saving: text.txt
Text has been saved successfully.

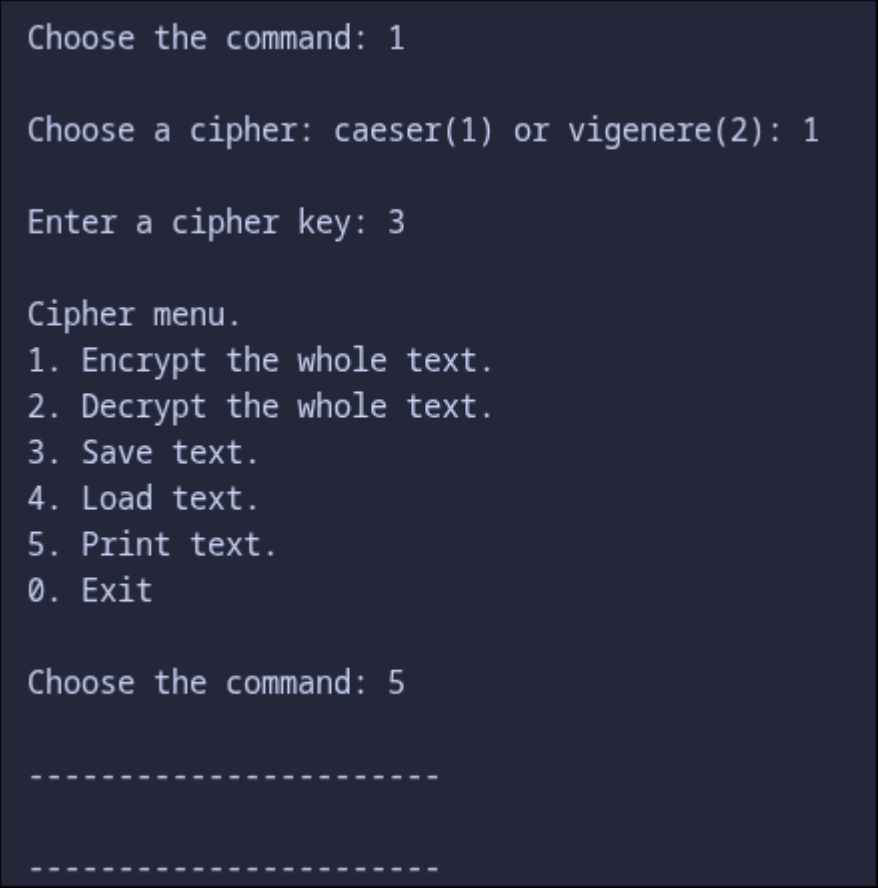
```

Scenario / Test Case	Expected Result
 <pre> Choose the command: 4 Enter the file name for loading: text.txt Text has been loaded successfully. Supported commands. 1. Append to end. 2. New line. 3. Save to file. 4. Load from file. 5. Print to console. 6. Insert. 7. Insert with replacement. 8. Change checklist. 9. Change contact. 10. Search. 11. Delete. 12. Delete line. 13. Copy. 14. Paste. 15. Cut. 16. Undo. 17. Redo. 18. Cipher menu. 0. Exit. Choose the command: 5 ----- Text: hello world [] to do Contact - name surname, Email: name@gmail.com ----- </pre>	
File Encryption Flow	Triggering encryption successfully invokes the dynamic DLL, obfuscates the text buffer, and saves it to a file.
Screenshot: <PUT HERE>	

Scenario / Test Case	Expected Result
<pre> Choose the command: 18 Cipher menu. 1. Encrypt the whole text. 2. Decrypt the whole text. 3. Save text. 4. Load text. 5. Print text. 0. Exit Choose the command: 1 Choose a cipher: caesar(1) or vigenere(2): 1 Enter a cipher key: 3 Cipher menu. 1. Encrypt the whole text. 2. Decrypt the whole text. 3. Save text. 4. Load text. 5. Print text. 0. Exit Choose the command: 3 Enter the file name for saving: encrypted.txt Text has been saved successfully. ≡ encrypted.txt 1 Text: koor zruog 2 [] wr gr 3 Contact - qdph vxuqdph, Email: qdph@jpdlo.frp 4 </pre>	
File Decryption Round-Trip	Loading and decrypting the obfuscated file

Scenario / Test Case	Expected Result
	restores the original mixed-type text editor contents completely.
Screenshot: <PUT HERE>  <pre> Choose the command: 4 Enter the file name for loading: encrypted.txt Text has been loaded successfully. Cipher menu. 1. Encrypt the whole text. 2. Decrypt the whole text. 3. Save text. 4. Load text. 5. Print text. 0. Exit Choose the command: 2 Choose a cipher: caeser(1) or vigenere(2): 1 Enter a cipher key: 3 Cipher menu. 1. Encrypt the whole text. 2. Decrypt the whole text. 3. Save text. 4. Load text. 5. Print text. 0. Exit Choose the command: 5 ----- Text: hello world [] to do Contact - name surname, Email: name@gmail.com ----- </pre>	

Edge Cases & Quality Acceptance Criteria

Scenario / Test Case	Expected Result
Empty Buffer Encryption	Attempting to encrypt an empty editor session generates a valid empty/header-only encrypted file without throwing exceptions or crashing.
Screenshot: <PUT HERE>  <pre> Choose the command: 1 Choose a cipher: caesar(1) or vigenere(2): 1 Enter a cipher key: 3 Cipher menu. 1. Encrypt the whole text. 2. Decrypt the whole text. 3. Save text. 4. Load text. 5. Print text. 0. Exit Choose the command: 5 ----- ----- </pre>	
Loading Corrupted / Invalid Encrypted File	Trying to decrypt a regular text file or a corrupted encrypted file handles the error gracefully via CLI, showing an explicit warning message.
Screenshot: <PUT HERE> // It just decrypts it as a normal encrypted file.	
CLI Input Validation	Passing missing or out-of-range arguments for row commands (e.g., missing contact email) outputs a clean, descriptive operational error.
Screenshot: <PUT HERE>	

Scenario / Test Case	Expected Result
<pre> Choose the command: 2 Choose line type: text (1), checklist(2), contact information (3): 3 Insert contact name and surname: Insert contact email: New line is started. Supported commands. 1. Append to end. 2. New line. 3. Save to file. 4. Load from file. 5. Print to console. 6. Insert. 7. Insert with replacement. 8. Change checklist. 9. Change contact. 10. Search. 11. Delete. 12. Delete line. 13. Copy. 14. Paste. 15. Cut. 16. Undo. 17. Redo. 18. Cipher menu. 0. Exit. Choose the command: 5 ----- Contact - , Email: ----- </pre>	
Memory Leak Validation	Running the application under Valgrind or AddressSanitizer shows 0 memory leaks upon creating, destroying polymorphic lines, and dynamically unloading the DLL.
Screenshot: <PUT HERE>	

Scenario / Test Case	Expected Result
	<pre>Choose the command: 0 ==435229== ==435229== HEAP SUMMARY: ==435229== in use at exit: 0 bytes in 0 blocks ==435229== total heap usage: 109 allocs, 109 frees, 111,651 bytes allocated ==435229== ==435229== All heap blocks were freed -- no leaks are possible ==435229== ==435229== For lists of detected and suppressed errors, rerun with: -s ==435229== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)</pre>